final_project

November 3, 2026

1 Final Project

1.1 Final Project PPDS Programming in Python for Data Science

1.1.1 Author Elizabeth Jasmi George

Introduction: This project is an exploratory data analysis of the given Disney movies dataset and finding answers to some intersting questions that comes to mind regarding disney movies. The dataset consists of 5 CSV files. Since the 'Data world' from which this data was collected is presently retired, I could not get the viewpoint of the person who collected this data. Looking at the csv files, Here is an elaboration of the kinds of the data. The values are separarted by commas in all the 5 files.

disney_movies_total_gross contains 6 columns [movie_title release_date genre MPAA_rating total_gross inflation_adjusted_gross] The disney_movies_total_gross dataset gives information of the title of the movie, movie release date, genre of the movie, self declated rating of the movie indicating appropriateness for audience, total gross collection, amount of collection adjusted with inflation rate.

disney_revenue_1991-2016 contains 7 columns ['Year', 'Studio Entertainment[NI 1]', 'Disney Consumer Products[NI 2]','Disney Interactive[NI 3][Rev 1]', 'Walt Disney Parks and Resorts', 'Disney Media Networks', 'Total'] The disney_revenue_1991-2016 dataset consisits of mostly numbers indicating the revenue of Disney from many operational divisions.

disney-characters contains 5 columns ['movie_title', 'release_date', 'hero', 'villian', 'song'] The disney_characters dataset contains details title of the movie, release date, name of the hero, name of the villain, and a popular song in the movie

disney-director contains 2 columns ['name', 'director'] The disney-director dataset contains only 2 columns indicating the name of the movie and the director of that movie.

disney-voice-actors contains 3 columns ['character', 'voice-actor', 'movie'] The disney-voice-actors dataset contains the name of the character, the name of the voice actor who gave sound for that character, and the name of the movie in which the character appears.

Since the disney_movies_total_gross.csv with 579 rows contain diverse kinds of data about a movie, and has a good proportion of non-null values I am using that dataset first to find answers to the following questions.

1. Which is the letter or digit with with most of the Disney movie names started? (why : Movie industry is said believe in numnerolgogy, I want to see if Disney follows any letter or just distributes names equally)

2. How many movies did disney produce in each genre? (Why: It is interesting to know the distribution of genres of movies from this famous animation organization)
3. In which Genre does most of Disney movies belong to? (Why: Following the previous question, it is interesting to know the most produced genre)
4. Plot the count of movies against genre in a bar plot to get an idea about genre distribution visually. (Why: To get a visual representation of the distribution of genres)
5. which movie collected least in the box office? (Why: want to see whether I have heard the name of that movie that collected the least)
6. which three movie collected the most in the box office? (Why: I wanted to knopw the movie that collected the most in box office, but in second thoughtI want to see which three movies came as top hits)
7. What is the Total Gross amount Disney made from movies in each year from 1937-2016? (Why: To get an idea of the ups and downs of animation movie industry)
8. Plot the Total gross amount against year to get a visual idea of the question above. (Why: to visualize the troughs and peaks of gross collection)
9. Plot the transition of maximum grossing amount and minimum grossing amount in years from 1937-2016 (Why: to see deeper into the years in terms of gross)

Now There are some questions I am curious about regarding the sound of the Disney characters

10. Which voice actors gave voice to the Heroes in Disney movies? (Why: We identify the characters a lot with the sound. I want to know who gave voice to Heroes)
11. How many heroes has each voice actor given sound to? (Why: To know whether there are voice actors extensively giving voices to characters in Disney movies)

out of curiosity I want to know who directed the most number of Disney movies. Because the dataset has only 2 columns, I am not exploring any other possibilties from that dataset

12. Who is the director who directed the most number of disney movies? (Why: To know the name of a very talented person)

coming back to the disney_movies_total_gross with the possibilites of extracting more from the datetime object I want to know the following information

13. Whether there is any popular day in a week for a Disney movie release? (Why: to test the popular undertanding of Friday releases)
14. Which is the second preferred day in the week that Disney selected for movie release? (Why: to know if there is another release day frequently coming)
15. What advice an I get on suitability for audience if there is a rating given for a movie? (Why: To know how to find and interpret the self declared movie rating)

To answer that question I wrote a python script with a function 'get_movie_rating' to get the MPAA rating of a movie and return the rating code and interpretation of the rating.

16. Which characters are contributing to the most gross collecting movies? (Why: know the characters which are popular among people and those whom people love to see in cinemas)
17. How many movies in a particular rating is produced by Disney? (Why: What is the distribution of a paricular rating in disnay movies)

To answer this question I added a function to the python script to group the movies by rating and return the count of a group

1.1.2 Start with importing the required libraries

Since this notebook covers data analysis and visualization, pandas library for data analysis and altair for visualization are imported

[1]:
```
#importing the library for data analysis
import pandas as pd
#import altair for data visualization
import altair as alt
```

2 Get the basic information about the datasets

Start by by reading the csv files to pandas dateframes with similar names using read_csv method. Since the delimiter is ' , ' setting that parameter is not required. use info and columns methods to know the basic structure and content type of the data

[2]:
```
Disney_revenue = pd.read_csv("data/disney_revenue_1991-2016.csv")
Disney_revenue.info()
Disney_revenue.columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26 entries, 0 to 25
Data columns (total 7 columns):
 #   Column                            Non-Null Count  Dtype
---  ------                            --------------  -----
 0   Year                              26 non-null     int64
 1   Studio Entertainment[NI 1]        25 non-null     float64
 2   Disney Consumer Products[NI 2]    24 non-null     float64
 3   Disney Interactive[NI 3][Rev 1]   12 non-null     float64
 4   Walt Disney Parks and Resorts     26 non-null     float64
 5   Disney Media Networks             23 non-null     object
 6   Total                             26 non-null     int64
dtypes: float64(4), int64(2), object(1)
memory usage: 1.6+ KB
```

[2]:
```
Index(['Year', 'Studio Entertainment[NI 1]', 'Disney Consumer Products[NI 2]',
       'Disney Interactive[NI 3][Rev 1]', 'Walt Disney Parks and Resorts',
       'Disney Media Networks', 'Total'],
      dtype='object')
```

Read the *disney-characters.csv* data to a date frame named *Disney_characters*

[3]:
```
# read the disney-characters.csv data to a date frame named Disney_characters
Disney_characters = pd.read_csv("data/disney-characters.csv")
Disney_characters.info()
Disney_characters.columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 56 entries, 0 to 55
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   movie_title   56 non-null     object
 1   release_date  56 non-null     object
 2   hero          52 non-null     object
 3   villian       46 non-null     object
 4   song          47 non-null     object
dtypes: object(5)
memory usage: 2.3+ KB
```

[3]:
```
Index(['movie_title', 'release_date', 'hero', 'villian', 'song'],
dtype='object')
```

Read the *disney-director.csv* data to a date frame named *Disney_directors*

[4]:
```
#Read the disney-director.csv data to a date frame named Disney_directors
Disney_directors = pd.read_csv("data/disney-director.csv")
Disney_directors.columns
```

[4]:
```
Index(['name', 'director'], dtype='object')
```

Read the *disney-voice-actors.csv* data to a date frame named *Disney_voice*

[5]:
```
#Read the disney-voice-actors.csv data to a date frame named Disney_voice
Disney_voice = pd.read_csv("data/disney-voice-actors.csv")
Disney_voice.columns
```

[5]:
```
Index(['character', 'voice-actor', 'movie'], dtype='object')
```

Read the *disney_movies_total_gross.csv* data to a date frame named *Disney_total_gross*

[6]:
```
# read the disney_movies_total_gross.csv and save it as a data frame␣
 ↪Disney_total_gross
Disney_total_gross = pd.read_csv("data/disney_movies_total_gross.csv")

# since this is the first dataframe I am using to answer questions, I need to␣
 ↪get a snapshot of the data
# using head() method
# find first 10 rows of the data
Disney_total_gross.head(10)
```

[6]:

	movie_title	release_date	genre	MPAA_rating	total_gross	inflation_adjusted_gross
0	Snow White and the Seven Dwarfs	Dec 21, 1937	Musical	G	$184,925,485	$5,228,953,251
1	Pinocchio	Feb 9, 1940	Adventure	G	$84,300,000	$2,188,229,052
2	Fantasia	Nov 13, 1940	Musical	G	$83,320,000	$2,187,090,808
3	Song of the South	Nov 12, 1946	Adventure	G	$65,000,000	$1,078,510,579
4	Cinderella	Feb 15, 1950	Drama	G	$85,000,000	$920,608,730
5	20,000 Leagues Under the Sea	Dec 23, 1954	Adventure	NaN	$28,200,000	$528,279,994
6	Lady and the Tramp	Jun 22, 1955	Drama	G	$93,600,000	$1,236,035,515
7	Sleeping Beauty	Jan 29, 1959	Drama	NaN	$9,464,608	$21,505,832
8	101 Dalmatians	Jan 25, 1961	Comedy	G	$153,000,000	$1,362,870,985
9	The Absent Minded Professor	Mar 16, 1961	Comedy	NaN	$25,381,407	$310,094,574

[7]:
```
#get information about the Disney_total_gross dataframe
Disney_total_gross.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 579 entries, 0 to 578
Data columns (total 6 columns):
 #   Column                   Non-Null Count  Dtype 
---  ------                   --------------  ----- 
 0   movie_title              579 non-null    object
 1   release_date             579 non-null    object
 2   genre                    562 non-null    object
 3   MPAA_rating              523 non-null    object
 4   total_gross              579 non-null    object
 5   inflation_adjusted_gross  579 non-null    object
dtypes: object(6)
memory usage: 27.3+ KB
```

[8]:
```
#get the names of columns in the Disney_total_gross dataframe
Disney_total_gross.columns
```

[8]:
```
Index(['movie_title', 'release_date', 'genre', 'MPAA_rating', 'total_gross',
       'inflation_adjusted_gross'],
      dtype='object')
```

[9]:
```
#find the data types of the columns in Disney_total_gross
Disney_total_gross.dtypes
```

[9]:
```
movie_title                object
release_date               object
genre                      object
MPAA_rating                object
total_gross                object
inflation_adjusted_gross   object
dtype: object
```

if I remove all the NaN or 0 values now, I may miss some information. So I am cleaning the data according to the question which I am trying to answer

2.0.1 Question 1. Which is the letter or digit with with most of the Disney movie names started?

[10]:
```
# find the letter or digit with which the most Disney movies started
Disney_total_gross["first_letter"] = Disney_total_gross['movie_title'].str.
 ↪strip().str[0].str.upper()

letter_counts = Disney_total_gross["first_letter"].value_counts()
most_common = letter_counts.idxmax()
most_common
```

[10]:
```
'T'
```

2.0.2 Answer 1. 'T' is the most frequently occuring first letter for Disney movies.

[11]:
```
# printing the first row to find how many times T is apperaing as the first␣
 ↪letter of Movie name
print(letter_counts.head(1))
```

```
first_letter
T    140
Name: count, dtype: int64
```

In my opinion the preference of Letter T as the first name of the movie, is also contibuted by the use of Definite Article "The" in movie titles. Since many of the disney movies are adaptions of stories, it is reasonable that the movie name will align with the story name, which in most cases starts with "The". But there are other titles to starting with T.

2.0.3 Question 2. How many movies did disney produce in each genre?

2.0.4 (Why: It is interesting to know the distribution of genres of movies from this famous animation organization)

[12]:
```
#find the number of disney movies in each genre
genre_df = Disney_total_gross['genre'].value_counts().reset_index()
genre_df
```

[12]:

	genre	count
0	Comedy	182
1	Adventure	129
2	Drama	114
3	Action	40
4	Thriller/Suspense	24
5	Romantic Comedy	23
6	Musical	16
7	Documentary	16
8	Western	7
9	Horror	6
10	Black Comedy	3
11	Concert/Performance	2

2.0.5 Question 3. In which Genre does most of Disney movies belong to?

2.0.6 (Why: Following the previous question, it is interesting to know the most produced genre)

[13]:
```
# find the genre of movies which most of the Disney productions belong to
genre_df.iloc[0]
```

[13]:
```
genre    Comedy
count       182
Name: 0, dtype: object
```

2.0.7 Answer 3. The most produced genre of movies is Comedy with 182 movies in that genre

2.0.8 Question 4. Plot the count of movies against genre in a bar plot to get an idea about genre distribution visually.

2.0.9 (Why: To get a visual representation of the distribution of genres)

[14]:
```
#plot the number of disney movies by genre
genre_chart = (alt.Chart(genre_df, width =500, height =300).mark_bar(color =␣
↪'Teal')
   .encode(x ='genre', y = 'count')
   .properties(title = "Fig 1. How many movies in each genre did Disney␣
↪produce")
```

```
            )
genre_chart
```

[14]:
```
alt.Chart(…)
```

In later cells I want to calculate the gross in each year, so I am converting the release date to a datetime object and extracting the year from it. I am keeping that extracted year in a separate column in the same data frame

[15]:
```
# extract the year from release_date and set it as another column
Disney_total_gross_with_year_column = (Disney_total_gross
    .assign(year=  pd.to_datetime(Disney_total_gross["release_date"]).dt.year))
Disney_total_gross_with_year_column
```

[15]:

	movie_title	release_date	genre	MPAA_rating	total_gross	inflation_adjusted_gross	first_letter	year
0	Snow White and the Seven Dwarfs	Dec 21, 1937	Musical	G	$184,925,485	$5,228,953,251	S	1937
1	Pinocchio	Feb 9, 1940	Adventure	G	$84,300,000	$2,188,229,052	P	1940
2	Fantasia	Nov 13, 1940	Musical	G	$83,320,000	$2,187,090,808	F	1940
3	Song of the South	Nov 12, 1946	Adventure	G	$65,000,000	$1,078,510,579	S	1946
4	Cinderella	Feb 15, 1950	Drama	G	$85,000,000	$920,608,730	C	1950
..	…	…	…	…	…	…	…	…
574	The Light Between Oceans	Sep 2, 2016	Drama	PG-13	$12,545,979	$12,545,979	T	2016
575	Queen of Katwe	Sep 23, 2016	Drama	PG	$8,874,389	$8,874,389	Q	2016
576	Doctor Strange	Nov 4, 2016	Adventure	PG-13	$232,532,923	$232,532,923	D	2016
577	Moana	Nov 23, 2016	Adventure	PG	$246,082,029	$246,082,029	M	2016
578	Rogue One: A Star Wars Story	Dec 16, 2016	Adventure	PG-13	$529,483,936	$529,483,936	R	2016

[579 rows x 8 columns]

The data types are of type object. We need to clean data by assigning the correct type (float) to total_gross after removing the $ sign and commas, so as to make plotting and calculations possible.

[16]:
```
# The data types are of type object. We need to clean data by assigning the␣
 ↪correct type (float) to total_gross
```

```
# after removing the $ sign and commas, so as to make plotting and calculations␣
 ↪possible

Disney_total_gross_with_year_column = (Disney_total_gross_with_year_column
    .assign(total_gross_number=␣
 ↪Disney_total_gross_with_year_column['total_gross']
        .replace('[$|,]','',regex=True).astype(float)))
Disney_total_gross_with_year_column
```

[16]:

	movie_title	release_date	genre	MPAA_rating	\
0	Snow White and the Seven Dwarfs	Dec 21, 1937	Musical	G	
1	Pinocchio	Feb 9, 1940	Adventure	G	
2	Fantasia	Nov 13, 1940	Musical	G	
3	Song of the South	Nov 12, 1946	Adventure	G	
4	Cinderella	Feb 15, 1950	Drama	G	
..	…	…	…	…	
574	The Light Between Oceans	Sep 2, 2016	Drama	PG-13	
575	Queen of Katwe	Sep 23, 2016	Drama	PG	
576	Doctor Strange	Nov 4, 2016	Adventure	PG-13	
577	Moana	Nov 23, 2016	Adventure	PG	
578	Rogue One: A Star Wars Story	Dec 16, 2016	Adventure	PG-13	

	total_gross	inflation_adjusted_gross	first_letter	year	\
0	$184,925,485	$5,228,953,251	S	1937	
1	$84,300,000	$2,188,229,052	P	1940	
2	$83,320,000	$2,187,090,808	F	1940	
3	$65,000,000	$1,078,510,579	S	1946	
4	$85,000,000	$920,608,730	C	1950	
..	…	…	…	…	
574	$12,545,979	$12,545,979	T	2016	
575	$8,874,389	$8,874,389	Q	2016	
576	$232,532,923	$232,532,923	D	2016	
577	$246,082,029	$246,082,029	M	2016	
578	$529,483,936	$529,483,936	R	2016	

	total_gross_number
0	184925485.0
1	84300000.0
2	83320000.0
3	65000000.0
4	85000000.0
..	…
574	12545979.0
575	8874389.0
576	232532923.0
577	246082029.0
578	529483936.0

```
[579 rows x 9 columns]
```

Sort the data based on the total gross and drop the rows where total gross is not available

[17]:
```
# sort the data based on the total gross and drop the rows where total gross is␣
↪not available
Disney_total_gross_with_year_column.sort_values("total_gross_number").dropna()
```

[17]:
```
                                movie_title  release_date        genre  \
511                                Zokkomon  Apr 22, 2011    Adventure
487                      Walt and El Grupo  Sep 10, 2009  Documentary
280  An Alan Smithee Film: Burn Hollywood …  Feb 27, 1998       Comedy
502       Gedo Senki (Tales from Earthsea)  Aug 13, 2010    Adventure
495                  Waking Sleeping Beauty  Mar 26, 2010  Documentary
..                                      …           …           …
558                Avengers: Age of Ultron   May 1, 2015       Action
571                           Finding Dory  Jun 17, 2016    Adventure
578           Rogue One: A Star Wars Story  Dec 16, 2016    Adventure
524                           The Avengers   May 4, 2012       Action
564   Star Wars Ep. VII: The Force Awakens  Dec 18, 2015    Adventure

    MPAA_rating   total_gross inflation_adjusted_gross first_letter  year  \
511          PG        $2,815                  $2,984            Z  2011
487          PG       $20,521                 $23,064            W  2009
280           R       $45,779                 $82,277            A  1998
502       PG-13       $48,658                 $51,988            G  2010
495          PG       $80,741                 $86,264            W  2010
..          …           …                       …          …   …
558       PG-13  $459,005,868            $459,005,868            A  2015
571          PG  $486,295,561            $486,295,561            F  2016
578       PG-13  $529,483,936            $529,483,936            R  2016
524       PG-13  $623,279,547            $660,081,224            T  2012
564       PG-13  $936,662,225            $936,662,225            S  2015

     total_gross_number
511              2815.0
487             20521.0
280             45779.0
502             48658.0
495             80741.0
..                  …
558         459005868.0
571         486295561.0
578         529483936.0
524         623279547.0
564         936662225.0
```

```
[513 rows x 9 columns]
```

2.0.10 Question 5. which movie collected least in the box office?

2.0.11 (Why: want to see whether I have heard the name of that movie that collected the least)

[18]:
```
# which movie collected least in the box office
Disney_total_gross_with_year_column.sort_values("total_gross_number").dropna().
 ↪iloc[0]
```

[18]:
```
movie_title                   Zokkomon
release_date                Apr 22, 2011
genre                          Adventure
MPAA_rating                           PG
total_gross                       $2,815
inflation_adjusted_gross          $2,984
first_letter                           Z
year                                2011
total_gross_number                2815.0
Name: 511, dtype: object
```

2.0.12 Answer 5. 'Zokkomon' Collected the least

2.0.13 Question 6. which three movie collected the most in the box office?

2.0.14 (Why: I wanted to knopw the movie that collected the most in box office, but in second thought I want to see which three movies came as top hits)

[19]:
```
# which three movie collected the most in the box office
Disney_total_gross_with_year_column.sort_values("total_gross_number",␣
 ↪ascending= False).dropna().iloc[0:3]
```

[19]:
```
                          movie_title  release_date      genre  \
564  Star Wars Ep. VII: The Force Awakens  Dec 18, 2015  Adventure
524                       The Avengers   May 4, 2012     Action
578          Rogue One: A Star Wars Story  Dec 16, 2016  Adventure

    MPAA_rating   total_gross inflation_adjusted_gross first_letter  year  \
564       PG-13  $936,662,225            $936,662,225            S  2015
524       PG-13  $623,279,547            $660,081,224            T  2012
578       PG-13  $529,483,936            $529,483,936            R  2016

     total_gross_number
564         936662225.0
524         623279547.0
578         529483936.0
```

2.0.15 Answer 6. Most collecting

2.0.16 Ist Position - Star Wars Ep. VII: The Force Awakens

2.0.17 2nd Position - The Avengers

2.0.18 3rd position - Rogue One: A Star Wars Story

2.0.19 Question 7. What is the Total Gross amount Disney made from movies in each year from 1937-2016?

2.0.20 (Why: To get an idea of the ups and downs of animation movie industry)

[20]:
```
#group the data frame by year and aggregrate sum of total_gross expressed as␣
 ↪number
Disney_total_gross_grouped_by_year= (Disney_total_gross_with_year_column.
 ↪groupby('year', as_index= False)
    .agg({"total_gross_number" :'sum'}))
Disney_total_gross_grouped_by_year
```

[20]:

	year	total_gross_number
0	1937	1.849255e+08
1	1940	1.676200e+08
2	1946	6.500000e+07
3	1950	8.500000e+07
4	1954	2.820000e+07
5	1955	9.360000e+07
6	1959	9.464608e+06
7	1961	1.885997e+08
8	1962	9.230769e+06
9	1963	2.218235e+07
10	1967	1.418430e+08
11	1968	2.154005e+07
12	1970	7.428275e+07
13	1971	1.787117e+07
14	1975	3.191650e+07
15	1977	1.027176e+08
16	1979	3.584190e+07
17	1980	1.390000e+07
18	1981	5.989923e+07
19	1982	3.861798e+07
20	1983	3.883373e+07
21	1984	7.099950e+07
22	1985	7.298163e+07
23	1986	2.545233e+08
24	1987	5.602088e+08
25	1988	4.791293e+08
26	1989	5.254473e+08
27	1990	6.138571e+08
28	1991	6.470503e+08

29	1992	8.545344e+08
30	1993	7.016521e+08
31	1994	1.092255e+09
32	1995	1.131964e+09
33	1996	1.134045e+09
34	1997	8.036171e+08
35	1998	1.229279e+09
36	1999	1.200292e+09
37	2000	1.098930e+09
38	2001	8.812468e+08
39	2002	1.159269e+09
40	2003	1.564114e+09
41	2004	1.157900e+09
42	2005	9.706468e+08
43	2006	1.427357e+09
44	2007	1.436788e+09
45	2008	9.682761e+08
46	2009	1.215143e+09
47	2010	1.518976e+09
48	2011	1.103207e+09
49	2012	1.452972e+09
50	2013	1.821352e+09
51	2014	1.514179e+09
52	2015	2.495663e+09
53	2016	2.873393e+09

2.0.21 Question 8. Plot the Total gross amount against year to get a visual idea of the question above.

2.0.22 (Why: to visualize the troughs and peaks of gross collection)

[21]:
```
#plot the total gross Disney made in every year in the data frame
chart_movienumber_vs_gross = (alt.Chart(Disney_total_gross_grouped_by_year,␣
 ↪width =800, height = 300).mark_bar()
    .encode(x="year", y= "total_gross_number")
    .properties(title =" Fig 2.Total Gross amount Disney made in years from␣
 ↪1937-2016")
                              )
chart_movienumber_vs_gross
```

[21]:
```
alt.Chart(…)
```

2.0.23 Question 9. Plot the transition of maximum grossing amount and minimum grossing amount in years from 1937-2016

2.0.24 (Why: to see deeper into the years in terms of gross)

[22]:
```
#group the data frame by year and aggregrate max amount of total_gross␣
 ↪expressed as number
Disney_max_gross_grouped_by_year= (Disney_total_gross_with_year_column.
 ↪groupby('year', as_index= False)
    .agg({"total_gross_number" :'max'}))
Disney_max_gross_grouped_by_year

#group the data frame by year and aggregrate min amount of total_gross␣
 ↪expressed as number
Disney_min_gross_grouped_by_year= (Disney_total_gross_with_year_column.
 ↪groupby('year', as_index= False)
    .agg({"total_gross_number" :'min'}))
Disney_min_gross_grouped_by_year
```

[22]:

	year	total_gross_number
0	1937	184925485.0
1	1940	83320000.0
2	1946	65000000.0
3	1950	85000000.0
4	1954	28200000.0
5	1955	93600000.0
6	1959	9464608.0
7	1961	10218316.0
8	1962	9230769.0
9	1963	22182353.0
10	1967	141843000.0
11	1968	21540050.0
12	1970	18607492.0
13	1971	17871174.0
14	1975	31916500.0
15	1977	0.0
16	1979	35841901.0
17	1980	2900000.0
18	1981	0.0
19	1982	4500000.0
20	1983	1204595.0
21	1984	8400000.0
22	1985	4100000.0
23	1986	4842778.0
24	1987	15212539.0
25	1988	4764606.0
26	1989	7210149.0
27	1990	1106475.0

28	1991	7300000.0
29	1992	2280148.0
30	1993	1044824.0
31	1994	60822.0
32	1995	293274.0
33	1996	34368.0
34	1997	590817.0
35	1998	45779.0
36	1999	178287.0
37	2000	1638202.0
38	2001	226792.0
39	2002	0.0
40	2003	1569918.0
41	2004	314000.0
42	2005	4710455.0
43	2006	4283255.0
44	2007	4600585.0
45	2008	225067.0
46	2009	20521.0
47	2010	48658.0
48	2011	2815.0
49	2012	12431792.0
50	2013	3254172.0
51	2014	17780194.0
52	2015	12429583.0
53	2016	7895708.0

[23]:
```
#clean the data omit values 0.0
Disney_max_gross_grouped_by_year =␣
 ↪Disney_max_gross_grouped_by_year[Disney_max_gross_grouped_by_year['total_gross_number']!
 ↪=0.0]
Disney_min_gross_grouped_by_year =␣
 ↪Disney_min_gross_grouped_by_year[Disney_min_gross_grouped_by_year['total_gross_number']!
 ↪=0.0]

#plot the transition of maximum grossing amount and minimum grossing amount in␣
 ↪years from 1937-2016
line_max_gross = (alt.Chart(Disney_max_gross_grouped_by_year, width =800,␣
 ↪height = 300).mark_line(color ="Green", opacity =0.5, size= 4)
    .encode(x="year", y = alt.Y("total_gross_number:Q" , title ="Max gross␣
 ↪amount")).properties(title =" Fig 3. Max Grossing vs minimum Grossing amount␣
 ↪Disney made in years from 1937-2016")
    )

line_min_gross = (alt.Chart(Disney_min_gross_grouped_by_year, width =800,␣
 ↪height = 300).mark_line(color ="Red", opacity =0.4, size= 2)
```

```
    .encode(x="year", y = alt.Y("total_gross_number:Q", title = "Min gross␣
 ↪amount")).properties(title =" Fig 3. Max Grossing vs minimum Grossing amount␣
 ↪Disney made in years from 1937-2016")
                )
line_max_gross + line_min_gross
```

[23]: `alt.LayerChart(…)`

The plot shows Maximum gross in years as a thick green line and minimum gross amount as thin red line

2.0.25 Question 10. Which voice actors gave voice to the Heroes in Disney movies?

2.0.26 (Why: We identify the characters a lot with the sound. I want to know who gave voice to Heroes)

[24]:
```
# Merge disney characters and voice actors data
Disney_characters_df = pd.read_csv("data/disney-characters.csv")
Disney_voice_actors_df = pd.read_csv("data/disney-voice-actors.csv")

#since movie_title and song contain new line character in many entries I want␣
 ↪to remove that
#and reassign to the column
Disney_characters_df = Disney_characters_df.assign(movie_title =␣
 ↪Disney_characters_df['movie_title'].str.strip(), song =␣
 ↪Disney_characters_df['song'].str.strip() )
hero_voices_df = (Disney_characters_df.merge(Disney_voice_actors_df, left_on =␣
 ↪"hero",
                            right_on = "character", how = 'inner').dropna(subset␣
 ↪=['character','voice-actor']))
hero_voices_df
```

[24]:

	movie_title	release_date	\
0	Snow White and the Seven Dwarfs	December 21, 1937	
1	Pinocchio	February 7, 1940	
3	Bambi	August 13, 1942	
4	Bambi	August 13, 1942	
5	Bambi	August 13, 1942	
6	Saludos Amigos	February 6, 1943	
7	The Three Caballeros	February 3, 1945	
8	Fun and Fancy Free	September 27, 1947	
9	Cinderella	February 15, 1950	
10	Alice in Wonderland	July 28, 1951	
11	Peter Pan	February 5, 1953	
12	Sleeping Beauty	January 29, 1959	
13	One Hundred and One Dalmatians	January 25, 1961	
14	The Sword in the Stone	December 25, 1963	
15	The Jungle Book	October 18, 1967	

16	Robin Hood	November 8, 1973
17	The Many Adventures of Winnie the Pooh	March 11, 1977
18	The Black Cauldron	July 24, 1985
19	Oliver & Company	November 18, 1988
20	The Little Mermaid	November 17, 1989
21	Beauty and the Beast	November 22, 1991
22	Aladdin	November 25, 1992
23	The Lion King	June 24, 1994
24	The Lion King	June 24, 1994
25	Pocahontas	June 23, 1995
26	The Hunchback of Notre Dame	June 21, 1996
27	Hercules	June 27, 1997
28	Mulan	June 19, 1998
29	Tarzan	June 18, 1999
30	Tarzan	June 18, 1999
31	Dinosaur	May 19, 2000
32	The Emperor's New Groove	December 15, 2000
33	Atlantis: The Lost Empire	June 15, 2001
34	Treasure Planet	November 27, 2002
35	Brother Bear	November 1, 2003
36	Home on the Range	April 2, 2004
37	Meet the Robinsons	March 30, 2007
38	Bolt	November 21, 2008
39	The Princess and the Frog	December 11, 2009
40	Tangled	November 24, 2010
41	Winnie the Pooh	July 15, 2011
42	Frozen	November 27, 2013
43	Zootopia	March 4, 2016

	hero	villian	song	\
0	Snow White	Evil Queen	Some Day My Prince Will Come	
1	Pinocchio	Stromboli	When You Wish upon a Star	
3	Bambi	Hunter	Love Is a Song	
4	Bambi	Hunter	Love Is a Song	
5	Bambi	Hunter	Love Is a Song	
6	Donald Duck	NaN	Saludos Amigos	
7	Donald Duck	NaN	NaN	
8	Mickey Mouse	Willie the Giant	NaN	
9	Cinderella	Lady Tremaine	Bibbidi-Bobbidi-Boo	
10	Alice	Queen of Hearts	The Unbirthday Song	
11	Peter Pan	Captain Hook	You Can Fly!	
12	Aurora	Maleficent	Once Upon a Dream	
13	Pongo	Cruella de Vil	Cruella De Vil	
14	Arthur	Madam Mim	Higitus Figitus	
15	Mowgli	Kaa and Shere Khan	The Bare Necessities	
16	Robin Hood	Prince John	Oo De Lally	
17	Winnie the Pooh	NaN	Winnie the Pooh	

```
18            Taran         Horned King                                NaN
19          Oliver               Sykes  Once Upon a Time in New York City
20           Ariel             Ursula                      Under the Sea
21           Belle             Gaston                       Be Our Guest
22         Aladdin              Jafar                A Whole New World
23           Simba               Scar                   Circle of Life
24           Simba               Scar                   Circle of Life
25       Pocahontas  Governor Ratcliffe              Colors of the Wind
26        Quasimodo       Claude Frollo           God Help the Outcasts
27         Hercules              Hades                 Go the Distance
28            Mulan            Shan Yu       I'll Make a Man Out of You
29           Tarzan            Clayton            You'll Be in My Heart
30           Tarzan            Clayton            You'll Be in My Heart
31           Aladar               Kron                              NaN
32           Kuzco               Yzma          My Funny Friend and Me
33      Milo Thatch    Commander Rourke        Where the Dream Takes You
34      Jim Hawkins         John Silver                 I'm Still Here
35            Kenai             Denahi           Look Through My Eyes
36           Maggie        Alameda Slim                              NaN
37            Lewis              Doris                 Little Wonders
38             Bolt          Dr. Calico           I Thought I Lost You
39            Tiana        Dr. Facilier                  Almost There
40          Rapunzel       Mother Gothel                I See the Light
41  Winnie the Pooh                 NaN                Winnie the Pooh
42             Elsa         Prince Hans                       Let It Go
43       Judy Hopps                 NaN                Try Everything

          character                voice-actor  \
0       Snow White           Adriana Caselotti
1        Pinocchio                Dickie Jones
3            Bambi              Donnie Dunagan
4            Bambi             Hardie Albright
5            Bambi            Alexander Gould
6      Donald Duck  Clarence Nash; Tony Anselmo
7      Donald Duck  Clarence Nash; Tony Anselmo
8     Mickey Mouse                 Walt Disney
9       Cinderella                Ilene Woods
10           Alice           Kathryn Beaumont
11       Peter Pan             Bobby Driscoll
12          Aurora                 Mary Costa
13           Pongo                 Rod Taylor
14          Arthur               David Lander
15          Mowgli           Bruce Reitherman
16      Robin Hood              Brian Bedford
17  Winnie the Pooh          Sterling Holloway
18           Taran             Grant Bardsley
19          Oliver              Joey Lawrence
```

20	Ariel	Jodi Benson
21	Belle	Paige O'Hara
22	Aladdin	Scott Weinger; Brad Kane
23	Simba	Jonathan Taylor Thomas
24	Simba	Matthew Broderick
25	Pocahontas	Irene Bedard; Judy Kuhn
26	Quasimodo	Tom Hulce
27	Hercules	Tate Donovan; Joshua Keaton
28	Mulan	Ming-Na Wen; Lea Salonga
29	Tarzan	Tony Goldwyn
30	Tarzan	Alex D. Linz
31	Aladar	D. B. Sweeney
32	Kuzco	David Spade
33	Milo Thatch	Michael J. Fox
34	Jim Hawkins	Joseph Gordon-Levitt
35	Kenai	Joaquin Phoenix
36	Maggie	Roseanne Barr
37	Lewis	Jordan Fry
38	Bolt	John Travolta
39	Tiana	Anika Noni Rose
40	Rapunzel	Mandy Moore
41	Winnie the Pooh	Sterling Holloway
42	Elsa	Idina Menzel
43	Judy Hopps	Ginnifer Goodwin

	movie
0	Snow White and the Seven Dwarfs
1	Pinocchio
3	Bambi
4	Bambi
5	Bambi
6	The Wise Little Hen
7	The Wise Little Hen
8	Steamboat Willie
9	Cinderella
10	Alice in Wonderland
11	Peter Pan
12	Sleeping Beauty
13	101 Dalmatians
14	Jungle Cubs
15	The Jungle Book
16	Robin Hood
17	Winnie the Pooh and the Honey Tree
18	The Black Cauldron
19	Oliver & Company
20	The Little Mermaid
21	Beauty and the Beast

```
22                            Aladdin
23                        The Lion King
24                        The Lion King
25                           Pocahontas
26          The Hunchback of Notre Dame
27                            Hercules
28                               Mulan
29                              Tarzan
30                              Tarzan
31                            Dinosaur
32             The Emperor's New Groove
33            Atlantis: The Lost Empire
34                      Treasure Planet
35                         Brother Bear
36                   Home on the Range
37                  Meet the Robinsons
38                                Bolt
39            The Princess and the Frog
40                              Tangled
41  Winnie the Pooh and the Honey Tree
42                              Frozen
43                            Zootopia
```

2.0.27 Question 11. How many heors has each voice actor given sound to?

2.0.28 (Why: To know whether there are voice actors extensively giving voices to characters in Disney movies)

[25]:
```
#how many hero voice has each voice actor given voice to
hero_voices_df.groupby('voice-actor')['movie_title'].count()
```

[25]:
```
voice-actor
Adriana Caselotti              1
Alex D. Linz                   1
Alexander Gould                1
Anika Noni Rose                1
Bobby Driscoll                 1
Brian Bedford                  1
Bruce Reitherman               1
Clarence Nash; Tony Anselmo    2
D. B. Sweeney                  1
David Lander                   1
David Spade                    1
Dickie Jones                   1
Donnie Dunagan                 1
Ginnifer Goodwin               1
Grant Bardsley                 1
```

```
Hardie Albright                1
Idina Menzel                   1
Ilene Woods                    1
Irene Bedard; Judy Kuhn        1
Joaquin Phoenix                1
Jodi Benson                    1
Joey Lawrence                  1
John Travolta                  1
Jonathan Taylor Thomas         1
Jordan Fry                     1
Joseph Gordon-Levitt           1
Kathryn Beaumont               1
Mandy Moore                    1
Mary Costa                     1
Matthew Broderick              1
Michael J. Fox                 1
Ming-Na Wen; Lea Salonga       1
Paige O'Hara                   1
Rod Taylor                     1
Roseanne Barr                  1
Scott Weinger; Brad Kane       1
Sterling Holloway              2
Tate Donovan; Joshua Keaton    1
Tom Hulce                      1
Tony Goldwyn                   1
Walt Disney                    1
Name: movie_title, dtype: int64
```

2.0.29 Answer 11. There are two voice actors who played 2 roles each

2.0.30 Question 12. Who is the director who directed the most number of disney movies?

2.0.31 (Why: To know the name of a very talented person)

[26]:
```
#read the director data into a data frame
Disney_director_df = pd.read_csv("data/disney-director.csv")
#merge the disney movie total gross data with the director data on moive_title␣
 ↪of left df and name on right df
movie_director_merged_df = (Disney_total_gross_with_year_column
    .merge(Disney_director_df, left_on ="movie_title", right_on = "name"))
```

[27]:
```
#find the director who directed the most number of disney movies
# use value counts to find how many rows are listing a person as director
director_counts = movie_director_merged_df.value_counts("director")
top_director_name = director_counts.idxmax()
top_director_name
```

[27]:
```
'Wolfgang Reitherman'
```

2.0.32 Answer 12. 'Wolfgang Reitherman' is the most listed name as director of Disney movies

2.0.33 Question 15. What advice an I get on suitability for audience if there is a rating given for a movie?

2.0.34 (Why: To know how to find and interpret the self declared movie rating)

Now I am importing my python script which contains two functions.

I am calling the first function for getting the movie rating and its interpretation.

[28]:
```
import importlib
#import the script with the function for finding the rating of a movie
import get_movie_rating as gmr
importlib.reload(gmr)

#Get the rating for the movie "Pinocchio" by calling the function␣
 ↪get_movie_rating from the script with that same name
gmr.get_movie_rating(Disney_total_gross_with_year_column, "Pinocchio",␣
 ↪"MPAA_rating")
```

[28]:
```
'G- General Audience'
```

Call the fuction with different parameters

[29]:
```
gmr.get_movie_rating(Disney_total_gross_with_year_column, "The Jungle Book",␣
 ↪"MPAA_rating")
```

[29]:
```
'Not Rated'
```

[30]:
```
gmr.get_movie_rating(Disney_total_gross_with_year_column, "Dumbo",␣
 ↪"MPAA_rating")
```

[30]:
```
'Movie not found'
```

[31]:
```
gmr.get_movie_rating(Disney_total_gross_with_year_column, "Aladdin",␣
 ↪"MPAA_rating")
```

[31]:
```
'G- General Audience'
```

[32]:
```
gmr.get_movie_rating(Disney_total_gross_with_year_column, "Guardians of the␣
 ↪Galaxy", "MPAA_rating")
```

[32]:
```
'PG-13- Parents Strongly Cautioned'
```

[33]:
```
gmr.get_movie_rating(Disney_total_gross_with_year_column, "Sister Act",␣
 ↪"MPAA_rating")
```

[33]:
```
'PG- parental guidance'
```

[34]:
```
gmr.get_movie_rating(Disney_total_gross_with_year_column, "Casanova",␣
↪"MPAA_rating")
```

[34]:
```
'R- Restricted'
```

Test the function by running the unit tests in test_get_movie_rating

[35]:
```
#import the test file and test the get_movie_rating function
import importlib
import test_get_movie_rating as tgmr
importlib.reload(tgmr)
importlib.reload(gmr)
tgmr.test_get_movie_rating()
```

I an interested to see whether there is any popular day in a week for a Disney movie release. So I want to use the data from disney_movies_total_gross.csv

2.0.35 Question 13. Whether there is any popular day in a week for a Disney movie release?

2.0.36 (Why: to test the popular undertanding of Friday releases)

Load the data with release date as a datetime object

[36]:
```
#load the data frame and set release_date as a datetime object
disney_movies_with_release_day = pd.read_csv("data/disney_movies_total_gross.
↪csv", parse_dates = ['release_date'])
```

Check the data types

[37]:
```
disney_movies_with_release_day.dtypes
disney_movies_with_release_day.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 579 entries, 0 to 578
Data columns (total 6 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   movie_title             579 non-null    object
 1   release_date            579 non-null    datetime64[ns]
 2   genre                   562 non-null    object
 3   MPAA_rating             523 non-null    object
 4   total_gross             579 non-null    object
 5   inflation_adjusted_gross  579 non-null    object
dtypes: datetime64[ns](1), object(5)
memory usage: 27.3+ KB
```

Assign a new column with the day of the week in it. The day of the week is extracted from the release_date which is of type datetime

[38]:
```
#assign a new column with the day of the week in it. The day of the week is␣
 ↪extracted from the release_date which is of type datetime
disney_movies_with_release_day = disney_movies_with_release_day.
 ↪assign(day_of_week = disney_movies_with_release_day['release_date'].dt.
 ↪day_name()).dropna(subset = 'day_of_week')
disney_movies_with_release_day
```

[38]:

	movie_title	release_date	genre	MPAA_rating	total_gross	inflation_adjusted_gross	day_of_week
0	Snow White and the Seven Dwarfs	1937-12-21	Musical	G	$184,925,485	$5,228,953,251	Tuesday
1	Pinocchio	1940-02-09	Adventure	G	$84,300,000	$2,188,229,052	Friday
2	Fantasia	1940-11-13	Musical	G	$83,320,000	$2,187,090,808	Wednesday
3	Song of the South	1946-11-12	Adventure	G	$65,000,000	$1,078,510,579	Tuesday
4	Cinderella	1950-02-15	Drama	G	$85,000,000	$920,608,730	Wednesday
..	…	…	…	…	…	…	…
574	The Light Between Oceans	2016-09-02	Drama	PG-13	$12,545,979	$12,545,979	Friday
575	Queen of Katwe	2016-09-23	Drama	PG	$8,874,389	$8,874,389	Friday
576	Doctor Strange	2016-11-04	Adventure	PG-13	$232,532,923	$232,532,923	Friday
577	Moana	2016-11-23	Adventure	PG	$246,082,029	$246,082,029	Wednesday
578	Rogue One: A Star Wars Story	2016-12-16	Adventure	PG-13	$529,483,936	$529,483,936	Friday

[579 rows x 7 columns]

[39]:
```
# group the entries by day day of the week, find the count and sort based on␣
 ↪the count
(disney_movies_with_release_day.groupby("day_of_week").count()
    .reset_index().sort_values(by ="release_date", ascending = False))
```

[39]:

	day_of_week	movie_title	release_date	genre	MPAA_rating	total_gross	inflation_adjusted_gross	day_of_week
0	Friday	464	464	452	424	464	464	464
5	Wednesday	75	75	72	68	75	75	75
3	Thursday	20	20	19	14	20	20	20
4	Tuesday	9	9	8	7	9	9	9
1	Saturday	7	7	7	6	7	7	7
2	Sunday	4	4	4	4	4	4	4

	inflation_adjusted_gross
0	464
5	75
3	20
4	9
1	7
2	4

[40]:
```
#find the day of the week with the most count
counts = disney_movies_with_release_day.groupby("day_of_week").size().
 ↪sort_values(ascending = False)
counts.index[0]
```

[40]: 'Friday'

2.0.37 Answer 13. Excellent! as expected most of the Disney movies were released on Friday, which is the most popular release date prior to the upcoming weekends. Now let us see which next day of the week did Disney movie release happen frequently, after Fridays.

Now which is the next most frquent release day?

2.0.38 Question 14. Which is the second preferred day in the week that Disney selected for movie release?

2.0.39 (Why: to know if there is another release day frequently coming)

[41]:
```
counts.index[1]
```

[41]: 'Wednesday'

2.0.40 Answer 14. It is Wednesdays that had next frequent releases. Looking at the disney_movies_with_release_day, it is clear.

There are some Disney characters who are my favourites. I want to see which characters contributed to the maximally grossing movies and plot them using squares. For that I need to merge the Disney_total_gross_with_year_column(the cleaned version with numerical total_gross amount) and Disney_characters_df

2.0.41 Question 16. Which characters are contributing to the most gross collecting movies?

2.0.42 (Why: know the characters which are popular among people and those whom people love to see in cinemas)

[42]:
```
#merge
character_gross_df = (Disney_total_gross_with_year_column
    .merge(Disney_characters_df, how ="inner", on = "movie_title")
    .sort_values('total_gross_number').loc[:,['total_gross_number','hero']])
#remove the 0.0 values
character_gross_df =␣
 ↪character_gross_df[character_gross_df['total_gross_number'] !=0.0].dropna()
character_gross_df
```

[42]:

	total_gross_number	hero
5	9464608.0	Aurora
12	21288692.0	Taran
6	22182353.0	Arthur
13	23605534.0	Basil
39	26692846.0	Winnie the Pooh
16	27931461.0	Bernard and Miss Bianca
30	38120554.0	Jim Hawkins
11	43899231.0	Tod and Copper
20	44342956.0	Mowgli
10	48775599.0	Bernard and Miss Bianca
14	49576671.0	Oliver
32	50026353.0	Maggie
8	55675257.0	Thomas and Duchess
28	84052762.0	Milo Thatch
1	84300000.0	Pinocchio
3	85000000.0	Cinderella
31	85336277.0	Kenai
27	89296573.0	Kuzco
4	93600000.0	Lady and Tramp
34	97822171.0	Lewis
23	99112101.0	Hercules
22	100138851.0	Quasimodo
36	104400899.0	Tiana
15	111543479.0	Ariel
35	114053759.0	Bolt
24	120620254.0	Mulan
33	135386665.0	Ace Cluck
26	137748063.0	Aladar
21	141579773.0	Pocahontas
7	141843000.0	Mowgli
29	145771527.0	Lilo and Stitch
25	171091819.0	Tarzan

```
0          184925485.0          Snow White
40         189412677.0               Ralph
38         200821936.0            Rapunzel
43         201151353.0          Cinderella
18         217350219.0             Aladdin
17         218951625.0               Belle
42         222527828.0         Hiro Hamada
46         246082029.0               Moana
37         334191110.0               Alice
44         341268248.0          Judy Hopps
45         364001123.0              Mowgli
41         400738009.0                Elsa
19         422780140.0               Simba
```

Plotting the characters are contributing to the most gross collecting movies. The position of the character in x axis indicate their contribution to gross collection

[43]:
```
#plot the gross vs character
hero_vs_gross = (alt.Chart(character_gross_df, width =700, height =300)
    .mark_circle(color = "navy", size =50).encode(x = 'hero' , y␣
 ↪='total_gross_number')
    .properties(title = "Fig 4. Heroes contributing to the gross amount"))
hero_vs_gross
```

[43]:
```
alt.Chart(…)
```

2.0.43 Answer 16. From the plot it is visible that Simba is the hero in the maximally grossing movie.

2.0.44 And the second runner up is Elsa. Obviously *Lion King* and *Frozen* are highly rated and all-time favourites.

2.0.45 Question 17. How many movies in a particular rating is produced by Disney?

2.0.46 (Why: What is the distribution of a paricular rating in disnay movies)

The python script has a function called **get_number_of_movies_with_a_rating** which will return the number of movies with a particular rating passed as argument (default argument is 'G')

[44]:
```
import importlib
importlib.reload(gmr)
# I am checking withput passing any arugument as rating string. The default␣
 ↪value is 'G'
#So It must return the number of Disney movies which are rated 'G'
gmr.get_number_of_movies_with_a_rating(Disney_total_gross_with_year_column,␣
 ↪'MPAA_rating')
```

[44]:
```
86
```

[45]:
```
#
gmr.get_number_of_movies_with_a_rating(Disney_total_gross_with_year_column,␣
↪'MPAA_rating', 'PG-13')
```

[45]: 145

Calling the unit test test_get_number_of_movies_with_a_rating for this function get_number_of_movies_with_a_rating

[46]:
```
tgmr.test_get_number_of_movies_with_a_rating()
```

The test cases pass

[47]:
```
!black get_movie_rating.py
```

```
All done!
1 file left unchanged.
```

[48]:
```
!flake8 test_get_movie_rating.py
```

```
test_get_movie_rating.py:7:80: E501
line too long (81 > 79 characters)
test_get_movie_rating.py:15:80: E501
line too long (88 > 79 characters)
test_get_movie_rating.py:23:80: E501
line too long (88 > 79 characters)
test_get_movie_rating.py:24:80: E501
line too long (84 > 79 characters)
test_get_movie_rating.py:25:80: E501
line too long (88 > 79 characters)
test_get_movie_rating.py:33:80: E501
line too long (88 > 79 characters)
test_get_movie_rating.py:43:80: E501
line too long (82 > 79 characters)
test_get_movie_rating.py:44:80: E501
line too long (86 > 79 characters)
test_get_movie_rating.py:45:80: E501
line too long (82 > 79 characters)
```

[49]:
```
!black test_get_movie_rating.py
```

```
All done!
1 file left unchanged.
```

[50]:
```
!flake8 test_get_movie_rating.py
```

```
test_get_movie_rating.py:7:80: E501
line too long (81 > 79 characters)
test_get_movie_rating.py:15:80: E501
line too long (88 > 79 characters)
test_get_movie_rating.py:23:80: E501
```

```
line too long (88 > 79 characters)
test_get_movie_rating.py:24:80: E501
line too long (84 > 79 characters)
test_get_movie_rating.py:25:80: E501
line too long (88 > 79 characters)
test_get_movie_rating.py:33:80: E501
line too long (88 > 79 characters)
test_get_movie_rating.py:43:80: E501
line too long (82 > 79 characters)
test_get_movie_rating.py:44:80: E501
line too long (86 > 79 characters)
test_get_movie_rating.py:45:80: E501
line too long (82 > 79 characters)
```

2.0.47 Discussion:

The answers to each of the question in the introduction of this notebook is given in the notbook itself. The expectations were met. Friday as the most frequent release date and 'Simba' as the most grossing movie character is expected. Missing the data corresponding to the star wars and avengers movie in the characters data set would have contributed to them not coming in the Fig 4 plot. The character dataset may be considering more animation characters and less real life characters. The analysis gives a better understanding of disney productions and popularity of characters. The proportion of inflation adjusted gross with total gross is interesting to explore further.

2.1 References

1. Sample project
2. Pandas documentation https://pandas.pydata.org/docs/

3. PPDS course videos https://www.youtube.com/watch?v=yz6Wwa2MkQA&t=1469s
4. Bing search results
5. Alatir documentation https://altair-viz.github.io/
6. Stack overflow

2.1.1 Thank You

[ ]: